

Methods for Creating AI Agents

🔊 Listen to reading 11 minutes



This reading provides a high-level overview of AI Agents, their core components, and different methods for creating them. You'll understand the difference between building an agent from scratch versus using a framework.

Objectives

By the end of this reading, you will be able to:

- Define an AI Agent and its core components.
- Compare building an agent from scratch with using a framework.
- Describe the general design process of an AI Agent.
- Design a Router Agent decides which agent should handle a given request.

Understanding AI Agents

An **AI Agent** is an autonomous system that perceives its environment and takes actions to achieve a specific goal. Unlike a simple program, it can reason and plan to complete complex tasks, such as breaking a goal of "booking a flight" into smaller steps.

General Design

Most AI Agents follow a similar high-level design with these components:

1. **Perception:** Gathers information from the environment (for example, user input and web data)
2. **Reasoning/Planning:** Determines the best action based on perceived information and its goal
3. **Memory:** Stores relevant information to maintain context and make informed decisions
4. **Action:** Executes the chosen action such as sending a response or using an API

This creates a continuous cycle: **Perceive** → **Reason** → **Act**.

Building Agents: From Scratch vs. Frameworks

There are two main approaches to creating an AI Agent.

The "From Scratch" Approach

Creating an agent **from scratch** means manually implementing every part of the agent lifecycle—without relying on an agent framework. In the **Lab: Building AI Agents from Scratch with Python**, this approach is intentionally used so you can clearly see how agents work internally.

This means you are responsible for coding the logic for each part of the agent's cycle, including how it takes in information, processes it, and decides on its next action. This approach gives you complete control over every detail of the agent's behavior and architecture.

Each agent is built using plain Python functions, with explicit logic for:

- Understanding user input
- Deciding what to do
- Producing a response

Agent 1: Weather Agent (From Scratch)

The Weather Agent is responsible for answering weather-related questions.

How it works in the lab

- **Perception:** Receives the user's query (for example, "What is the weather in Paris?")
- **Reasoning:**

- Determines whether weather data needs to be fetched.
- Checks if previous weather data for the same city exists in memory.
- **Action:**
 - Calls the OpenWeather API.
 - Stores the retrieved weather data in memory.
 - Constructs a natural-language response for the user.

Example Code:

```

1 class WeatherAgent:
2     def __init__(self, api_key, memory_agent):
3         self.api_key = api_key
4         self.memory = memory_agent
5         self.url = "http://api.openweathermap.org/data/2.5/weather"
6
7     def answer(self, city):
8         params = {
9             "q": city,
10            "appid": self.api_key,
11            "units": "metric"
12        }
13
14        res = requests.get(self.url, params=params)
15
16        if res.status_code != 200:
17            return "I couldn't retrieve the weather right now."
18
19        data = res.json()
20
21        previous = self.memory.recall(city)
22        self.memory.store(city, data["main"])
23
24        response = (
25            f"The current weather in {city} is {data['weather'][0]['description']} "
26            f"with a temperature of {data['main']['temp']}°C."
27        )
28
29        if previous:
30            response += f" Earlier it was {previous['temp']}°C."
31
32        return response

```

Here, this example demonstrates a **"from scratch"** approach, where all core agent behaviors are implemented explicitly in the code. The agent manually handles the API interaction to fetch weather data, explicitly manages memory by storing and recalling previous weather information, and constructs the final response using developer-defined logic. Since no agent framework is used, the entire flow—from data retrieval to contextual response generation—is fully controlled and transparent, allowing everyone to clearly understand how each component of the agent operates.

Agent 2: Daily Dish Agent (From Scratch)

Here, is one more agent used in the lab: **The Daily Dish Agent**, which handles food-related requests.

The Daily Dish Agent handles food-related requests based on the restaurant information provided in the PDF, such as menu items, meal types, and general dining questions.

How It Works in the Lab

- **Perception:** The agent reads the user's request, for example:
 - "Do you have any daily specials?"
 - "What is your most popular dish?"
 - "Do you have vegetarian options?"
- **Reasoning:** The agent analyzes the intent of the request:
 - Meal type (breakfast, lunch, dinner)
 - Popular dishes
 - Dietary preferences (vegetarian, vegan)
- **Action:** The agent responds with an appropriate recommendation or answer based on the predefined restaurant data.

Example Code:

```

1 class DailyDishAgent:
2     def __init__(self, questions, answers):
3         self.answers = answers

```

```

3         self.answers = answers
4         self.vectorizer = TfidfVectorizer(
5             stop_words="english",
6             ngram_range=(1, 2)
7         )
8         self.doc_vectors = self.vectorizer.fit_transform(questions)
9
10    def answer(self, query):
11        query_vector = self.vectorizer.transform([query])
12        similarities = cosine_similarity(query_vector, self.doc_vectors)[0]
13        best_idx = np.argmax(similarities)
14
15        if similarities[best_idx] < 0.08:
16            return None
17
18        return self.answers[best_idx]

```

Here, this implementation is considered as **"from scratch"** because all aspects of the agent's behavior are explicitly defined by the developer. The FAQ data is manually sourced from the provided PDF, and the text vectorization and similarity comparison logic are implemented directly using TF-IDF and cosine similarity. No conversational AI or retrieval framework is involved, and key parameters such as similarity thresholds and response selection are fully controlled within the code. This design allows learners to clearly understand how semantic matching works internally and how meaningful responses are derived from structured data.

Agent Routers: Connecting Multiple Agents

In advanced systems, **multiple specialized agents** are used to handle different types of user queries, such as a **Weather Agent**, a **Daily Dish Agent**, or a **News Agent**. Each agent is designed to focus on a specific domain.

A **Router Agent** is responsible for deciding which agent should process an incoming request. This decision is made by analyzing the user's input and identifying relevant keywords.

In the lab, the router is implemented from scratch using simple conditional logic. If the user query contains the word **"weather"**, it is routed to the **Weather Agent**. If it contains **"food"**, it is routed to the **Daily Dish Agent**. If no matching keywords are found, the request is sent to a default agent.

Router Example:

```

1 def router(user_query):
2     if "weather" in user_query:
3         return weather_agent
4     elif "food" in user_query:
5         return daily_dish
6     else:
7         return default_agent

```

This routing logic is explicitly coded in the lab, as shown in the example function, and demonstrates **manual agent orchestration**. By writing the routing rules, keyword checks, and fallback behavior directly in code, to gain a clear understanding of how **multi-agent** systems coordinate internally without relying on any external agent framework.

The "Framework" Approach

Using a **framework** involves leveraging a pre-existing library or toolkit specifically designed to streamline the creation of AI Agents. These frameworks provide pre-built modules and abstractions for common functionalities such as orchestrating agent workflows, managing memory, and connecting to various tools and APIs. By using a framework, you can focus on defining the agent's unique purpose and logic rather than building the foundational infrastructure from the ground up.

How the Framework Approach Works by Component

- **Perception:** Frameworks handle user input and message parsing automatically.
- **Reasoning:** Reasoning is often delegated to built-in planners or language models, reducing the need for manual decision logic.
- **Memory:** Memory management is handled through provided abstractions such as conversation memory or vector stores.
- **Action:** Tools and APIs are registered once and invoked automatically by the agent when needed.

Framework Example:

```

1 agent = Agent(
2     tools=[weather_tool],
3     memory=ConversationMemory()
4 )
5
6 agent.run("What is the weather in London?")

```

Here, in this **"framework-based approach"**, the framework automatically handles input processing, memory management, tool invocation, and response generation. This results in faster development, cleaner code, and easier management of multi-agent systems. However, it offers less low-level control and requires familiarity with framework-specific concepts.

From Scratch vs Framework Comparison

Aspect	From Scratch (Weather + Daily Dish + Router in Lab)	Framework-Based Agent Implementation
Agent Design	Each agent (WeatherAgent, DailyDishAgent) is implemented as a standalone class with explicitly defined logic for perception, reasoning, and action.	Agents are typically declared using high-level abstractions where perception, reasoning, and action are handled internally by the framework.
Control	Full control over agent behavior: API calls, similarity thresholds, memory storage, and response formatting are all manually coded.	Control is abstracted; developers configure behavior through framework components rather than implementing each step directly.
Routing	A custom router function manually inspects user queries and decides whether to invoke the Weather Agent or Daily Dish Agent.	Routing is usually handled by built-in router or planner modules that automatically select tools or agents.
Memory Handling	Memory is explicitly passed and managed (for example, storing and recalling previous weather data).	Memory is managed by framework-provided memory objects (conversation memory, vector memory, etc.).
Complexity	Requires more boilerplate code, including manual keyword checks, API handling, and similarity calculations.	Significantly reduces boilerplate by providing pre-built orchestration and tool-calling mechanisms.
Learning Value	Excellent for understanding how agent internals work, including routing decisions, semantic matching, and memory usage.	Better suited for rapid development once agent concepts are already understood.
Flexibility	Highly flexible—custom logic, thresholds, and routing strategies can be implemented without constraints.	Flexibility is bounded by the framework's design patterns and supported features.
Debugging	Easier to trace issues since every step (routing, API call, similarity score) is visible and explicit.	Debugging may require deeper knowledge of the framework's internal abstractions.
Scalability	Scaling to many agents or complex workflows requires additional custom routing and coordination code.	Designed to scale naturally to multi-agent systems with minimal additional code.
Maintenance	The developer is responsible for maintaining and updating all components.	Framework updates often handle improvements and compatibility changes automatically.

Summary

This reading introduced you to AI Agents, their core design components, and the key differences between building them from scratch versus leveraging a framework along with Router agent.

Go to next item

✔ Completed